

Manual Técnico

Liquidaciones: Pago y Cobro

fontana_movimientos · actualizado el 17 de septiembre de 2026

Este documento explica cómo funciona el sistema de Liquidaciones (módulo liquidaciones) incluyendo la distinción entre Pago (Fontana le paga a la entidad) y Cobro (la entidad le paga a Fontana), y cómo esa dirección se representa en cada tipo de ítem que puede formar parte de una liquidación: Comprobantes, Movimientos de Caja, Retenciones y Retenciones INYM. Está pensado como referencia para modificar el código a mano en el futuro: qué campo mirar, qué función tocar, y qué otras partes del sistema hay que actualizar en conjunto.

1. Qué es una Liquidación

Una Liquidacion (tabla liquidacion, app liquidaciones) agrupa un conjunto de ítems -- Comprobantes, Movimientos de Caja, Retenciones y Retenciones INYM -- vinculados a UNA entidad (la contraparte; nunca Fontana). Cada ítem se marca a mano, al armar la liquidación, como parte del debe o del haber. Los totales debe y haber de la liquidación se recalculan siempre desde esas relaciones (nunca se cargan a mano) y $diferencia = debe - haber$.

1.1 Campos principales

Campo	Tipo	Qué significa
id	IntegerField (PK)	Manual, MAX(id)+1 (tabla sin AUTO_INCREMENT)
numero	CharField	Por defecto 'LIQ-<id>' si no se cargó a mano
fecha	DateField	Fecha de la liquidación
entidad	FK → Entidad	La contraparte -- agnóstica de dirección, igual para pago y cobro
tipo	CharField ('pago'/'cobro')	NUEVO (17/09/2026). Default 'pago'. Ver sección 2.
debe / haber	DecimalField	Calculados por recalcular_totales() -- ver sección 7

2. La distinción Pago vs. Cobro

Hasta septiembre de 2026 el sistema solo contemplaba una dirección: Fontana paga a la entidad (por ejemplo, liquidar lo que se le debe a un proveedor). El campo `Liquidacion.tipo` agrega la dirección opuesta: la entidad le paga a Fontana, para poder vincular un recibo/cobro con la factura que Fontana emitió.

tipo	Significado	Ejemplo típico
'pago' (default)	Fontana le paga a la entidad	Pagarle a un proveedor una factura de compra

tipo	Significado	Ejemplo típico
'cobro'	La entidad le paga a Fontana	Registrar el cobro de una factura de venta que Fontana emitió

Reglas importantes:

- El tipo se elige solo al crear la liquidación (por GET ?tipo=... o por el <select> del paso 1 del formulario). En edición queda fijo al valor ya guardado -- cambiarlo dejaría inconsistentes los ítems que ya se vincularon para esa dirección.
- El backend lo hace cumplir en `liquidacion_form()`: el POST solo aplica `tipo_post` cuando `liquidacion is None` (alta).
- Determina qué ítems ofrece `_armar_items()` (sección 5) y qué textos usan la pantalla, el PDF y el Excel (sección 8).

3. Cómo cada ítem representa la dirección

Ningún campo de emisor/receptor se agregó a Liquidacion en sí -- Liquidacion.entidad sigue siendo "la contraparte", sin importar la dirección. En cambio, cada tipo de ítem ya tenía (o ahora tiene) su propio mecanismo para saber quién es el emisor y quién el receptor, y liquidaciones simplemente lo consulta con el filtro que corresponde según el tipo de la liquidación.

Constante usada en todos los módulos:

```
ENTIDAD_PROPIA_ID = 100 # Entidad "Fontana"
(definida en comprobantes.views, movimientos_caja.views,
liquidaciones.views, retenciones.views y retenciones_inym.views)
```

3.1 Comprobante -- campo es_emisor

Tabla comprobante. entidad_emisor (FK, columna id_entidad) SIEMPRE guarda la contraparte; es_emisor dice de qué lado está:

es_emisor	Significa	Se usa en
1 o NULL (default)	La entidad emitió el comprobante (Fontana lo recibió) -- factura de COMPRA	tipo='pago'
0	Fontana emitió el comprobante (la entidad lo recibió) -- factura de VENTA	tipo='cobro'

Filtro en `_armar_items()`:

```
# tipo='cobro'
Comprobante.objects.filter(entidad_emisor=entidad, es_emisor=0)

# tipo='pago'
Comprobante.objects.filter(entidad_emisor=entidad).filter(
    Q(es_emisor=1) | Q(es_emisor__isnull=True)
)
```

3.2 MovimientoCaja -- receptor + emisor_relacion

Tabla movimiento_caja. receptor es un FK obligatorio (históricamente, siempre la contraparte de un pago saliente). emisor es una property que devuelve Fontana (id 100) por defecto, salvo que exista un

registro en la tabla lateral movimiento_caja_emisor (MovimientoCajaEmisor, relación 1 a 1 vía emisor_relacion).

Dirección	receptor	emisor_relacion.id_entidad
pago (saliente)	entidad	(sin registro -- emisor por defecto = Fontana)
cobro (entrante / recibo)	Fontana (100)	entidad

Este es el mismo mecanismo que ya usa el comando de gestión mover_receptor_a_emisor para corregir movimientos cargados con la dirección incorrecta.

```
# tipo='cobro'
MovimientoCaja.objects.filter(
    emisor_relacion__id_entidad=entidad, receptor_id=ENTIDAD_PROPIA_ID,
)

# tipo='pago'
MovimientoCaja.objects.filter(receptor=entidad)
```

3.3 Retencion -- campo es_emisor (NUEVO 17/09/2026)

Antes de este cambio, Retencion solo modelaba "Fontana le practicó una retención a la entidad" -- el módulo entero (formulario, PDF, Excel) está construido para que Fontana EMITA una "Constancia de Retención" (documento con firma) a un Proveedor. Se agregó el campo es_emisor, con el mismo criterio y los mismos choices que Comprobante.es_emisor, para poder registrar también la dirección contraria: una retención que la entidad le practicó a Fontana al pagarle ("retención sufrida").

es_emisor	Significa	Documento
1 o NULL (default)	Fontana practicó/emitió la retención (comportamiento histórico)	CONSTANCIA DE RETENCIÓN, con firma. Entidad = "Proveedor"
0	La entidad le practicó la retención a Fontana ("sufrida")	RETENCIÓN RECIBIDA (registro interno), sin firma. Entidad = "Cliente"

Migración: retenciones/migrations/0002_retencion_es_emisor.py (ALTER TABLE con DEFAULT 1, hace backfill automático -- las retenciones ya cargadas quedan como "practicada", sin cambiar su significado).

Formulario: RetencionHeaderForm.es_emisor (retenciones/forms.py) -- select "Dirección" en el paso de alta/modificación, en retencion_form.html.

Textos de PDF/Excel: retenciones/views.py::_textos_retencion(es_emisor) y el dict _TEXTOS_POR_DIRECCION.

```
# liquidaciones/views.py::_armar_items -- tipo='cobro'
Retencion.objects.filter(entidad=entidad,
    es_emisor=Retencion.NO_ES_EMITOR)

# tipo='pago'
Retencion.objects.filter(entidad=entidad).filter(
    Q(es_emisor=Retencion.ES_EMITOR) | Q(es_emisor__isnull=True)
)
```

3.4 RetencionInym -- operador_emisor / operador retenido (sin cambio de esquema)

A diferencia de Retencion, la tabla retencion_inym YA tenía dos operadores (Inym_Operador): operador_emisor (quién practicó la retención) y operador retenido (a quién se la retuvieron) -- así vienen los datos que exporta el portal de INYM (columnas IDOPERADOR / OPER_RETENIDO, ver retenciones_inym/importador.py). En los datos reales YA existían casos con Fontana como operador retenido (confirmado por retenciones_inym/views.py, que ya excluía a Fontana del ranking filtrando operador retenido__entidad_id=ENTIDAD_PROPIA_ID). No hizo falta ninguna migración: solo se ajustó _armar_items() para filtrar por la dirección que corresponde al tipo de la liquidación.

Dirección	operador_emisor.entidad	operador retenido.entidad
pago (Fontana practicó)	Fontana	entidad
cobro (la entidad practicó)	entidad	Fontana

```
# tipo='cobro'
RetencionInym.objects.filter(
    operador_emisor__entidad=entidad,
    operador retenido__entidad_id=ENTIDAD_PROPIA_ID,
)

# tipo='pago'
RetencionInym.objects.filter(
    operador retenido__entidad=entidad,
    operador_emisor__entidad_id=ENTIDAD_PROPIA_ID,
)
```

Nota: las Retenciones y Retenciones INYM no tienen todavía un mecanismo de "otros" (agregar un ítem de otra entidad a mano) -- ese buscador (item_sin_liquidar_buscar) solo contempla Comprobantes y Movimientos de Caja.

4. La función central: _armar_items()

Ubicación: liquidaciones/views.py

```
def _armar_items(entidad, tipo=Liquidacion.TIPO_PAGO,
liquidacion_actual=None):
```

Devuelve, para la entidad y el tipo (pago/cobro) dados, los ítems sin liquidar de esa entidad más -- en modo edición (liquidacion_actual) -- los que ya están vinculados a esa liquidación, marcados como seleccionados:

- movimientos -- lista de MovimientoCaja candidatos, filtrados según la sección 3.2
- comprobantes -- lista de Comprobante candidatos, filtrados según la sección 3.1
- retenciones -- lista de Retencion candidatas, filtrados según la sección 3.3
- retenciones_inym -- lista de RetencionInym candidatas, filtradas según la sección 3.4
- movimientos_otros / comprobantes_otros -- en edición, ítems ya vinculados a ESTA liquidación pero que pertenecen a OTRA entidad (agregados a mano desde el buscador "Otros movimientos/comprobantes")

Cada ítem de cada lista se enriquece en memoria con tres atributos, antes de pasar al template:

Atributo	De dónde sale
monto_mostrar	_monto_item(item) -- .monto para MovimientoCaja, .total (convertido por tipo de cambio si aplica) para el resto
tipo_actual	'debe'/'haber' si ya está vinculado a esta liquidación, si no None
seleccionado	True si ya está vinculado (para marcar el checkbox al editar)

En modo edición, la exclusión de "otros" también es sensible a la dirección (mismo criterio que el filtro principal): si no lo fuera, un ítem vinculado a una liquidación de cobro podría desaparecer de la pantalla de edición en vez de listarse en su lista principal.

5. Alta / edición: liquidacion_form()

Ubicación: liquidaciones/views.py. Una sola vista para alta (pk=None) y edición (pk=<id>).

- Paso 1 (GET, formulario #form-seleccion): elegir tipo + entidad -- recarga la página con los ítems correspondientes.
- Paso 2 (POST, formulario #form-items): tildar ítems y elegir, por cada uno, si va al debe o al haber (mov_tipo_<id>, comp_tipo_<id>, etc. -- ver CONFIG_ITEMS).
- Al guardar: crea (o, en edición, borra y vuelve a crear) los LiquidacionMovimiento/LiquidacionComprobante/LiquidacionRetencion/LiquidacionRetencionInym correspondientes, y llama liquidacion.recalcular_totales().

El tipo de la liquidación se decide así:

```
tipo_liquidacion = liquidacion.tipo if liquidacion else
Liquidacion.TIPO_PAGO
# ... en alta (liquidacion is None):
# POST: tipo_post = request.POST.get('tipo') -- se aplica si es válido
# GET: tipo_get = request.GET.get('tipo') -- idem
# en edición: tipo_liquidacion queda SIEMPRE en liquidacion.tipo (no se
toca)
```

6. recalcular_totales()

Ubicación: liquidaciones/models.py, método de Liquidacion. Única fuente de verdad para debe/haber -- ninguna vista los suma a mano.

- Recorre las 4 relaciones (movimientos, comprobantes, retenciones, retenciones_inym) y suma, por separado, los montos de los ítems marcados "debe" y "haber" (Sum + Case a nivel SQL).
- Comprobantes: aplica el tipo de cambio (comprobante_tipo_de_cambio) si el comprobante está en moneda distinta a pesos, e invierte el signo si el tipo de comprobante contiene "nota de credito" (mismo criterio que liquidacion_diferencias).
- Redondea a 2 decimales y guarda con update_fields=['debe', 'haber'].
- diferencia (property) = debe - haber.

Esta función es agnóstica del tipo (pago/cobro) de la liquidación: solo mira el campo "tipo" (debe/haber) de cada ítem ya vinculado, no de dónde vino.

7. Textos de pantalla, PDF y Excel

7.1 Liquidación (PDF/Excel de una liquidación individual)

Ubicación: `liquidaciones/documentos.py`, `dict _TEXTOS_POR_TIPO` y función `_textos(liquidacion)`.

	tipo='pago'	tipo='cobro'
entidad_label	Proveedor	Cliente
debe_titulo	Comprobantes a Pagar	Comprobantes a Cobrar
debe_total_pdf/excel	Total a Pagar	Total a Cobrar
haber_titulo	Pago	Cobro
haber_total_pdf/excel	Total de Pago	Total de Cobro

Se usa en `generar_pdf_liquidacion()` y `generar_excel_liquidacion()` (hoja Resumen). El listado de ítems del debe/haber (`_filas_debe/_filas_haber`) es agnóstico del tipo -- solo lista lo que ya está vinculado.

7.2 Retención (Constancia / Registro interno)

Ubicación: `retenciones/views.py`, `dict _TEXTOS_POR_DIRECCION` y función `_textos_retencion(es_emisor)`. Mismo patrón que 7.1, aplicado a `retencion_pdf()` / `retencion_excel()`.

	es_emisor=1 (practicada)	es_emisor=0 (sufrida)
entidad_label	Proveedor	Cliente
titulo	CONSTANCIA DE RETENCIÓN	RETENCIÓN RECIBIDA (registro interno)
mostrar_firma	Sí (con línea de firma)	No (nota aclaratoria en su lugar)

8. Checklist: agregar un nuevo tipo de ítem a Liquidación

Si en el futuro se necesita sumar otro tipo de ítem (además de comprobantes, movimientos, retenciones y retenciones INYM), hay que tocar estos puntos:

- Modelo intermedio nuevo, mismo patrón que `LiquidacionMovimiento`: FK al ítem, FK a `Liquidacion`, campo `tipo` (`CharField 'debe'/'haber'`).
- Si el ítem va a necesitar distinguir pago/cobro, agregarle su propio campo o mecanismo de dirección (`es_emisor` si es un modelo con una sola entidad relacionada, o un par emisor/receptor si son dos) -- no agregarlo a `Liquidacion`.
- `_armar_items()` (`liquidaciones/views.py`): sumar el nuevo queryset, con su rama pago/cobro si corresponde.
- `Liquidacion.recalcular_totales()` (`liquidaciones/models.py`): agregar la nueva relación a la lista fuentes.
- `liquidaciones/documentos.py`: sumarlo en `_filas_debe/_filas_haber`.

- CONFIG_ITEMS en liquidaciones/views.py y el checkbox+select correspondiente en form.html.
- Si corresponde, sumarlo también al buscador de "otros" (item_sin_liquidar_buscar) y a los 4 listados "sin liquidar".

9. Referencia rápida de archivos

Archivo	Qué contiene
liquidaciones/models.py	Liquidacion (incl. tipo), LiquidacionComprobante/Movimiento/Retencion/RetencionInym, recalcular_totales()
liquidaciones/views.py	_armar_items, liquidacion_form, liquidacion_list, exportaciones "sin liquidar", rankings
liquidaciones/documentos.py	PDF/Excel de una liquidación individual (_textos, generar_pdf_liquidacion, generar_excel_liquidacion)
liquidaciones/forms.py	LiquidacionSeleccionForm (incluye tipo), formularios de reporte/ranking/filtro
liquidaciones/templates/liquidaciones/form.html	Formulario de alta/edición (paso 1: tipo+entidad, paso 2: ítems)
liquidaciones/templates/liquidaciones/list.html	Listado, con columna Tipo
liquidaciones/migrations/0002_liquidacion_tipo.py	Migración que agrega Liquidacion.tipo
retenciones/models.py	Retencion (incl. es_emisor), RetencionTipoImpuesto, RetencionTipoRegimen
retenciones/views.py	Alta/modificación/listado, _textos_retencion, retencion_pdf/excel (Constancia / Registro interno)
retenciones/forms.py	RetencionHeaderForm (incluye es_emisor), RetencionRenglonFormSet
retenciones/migrations/0002_retencion_es_emisor.py	Migración que agrega Retencion.es_emisor
retenciones_inym/models.py	RetencionInym (operador_emisor/operador_retenido -- sin cambios), InymRetencionTipo
retenciones_inym/importador.py	Importa el Excel de INYM; documenta el significado real de emisor/retenido
comprobantes/models.py	Comprobante.es_emisor (patrón que inspiró Retencion.es_emisor)
movimientos_caja/models.py	MovimientoCaja.receptor / .emisor (property), MovimientoCajaEmisor

10. Historial de cambios de este documento

Fecha	Cambio
17/09/2026	Versión inicial: se agrega Liquidacion.tipo (pago/cobro), Retencion.es_emisor, y se hace tipo-aware a _armar_items() para los 4 tipos de ítem.